CS4423 - Networks

Prof. Götz Pfeiffer
School of Mathematics, Statistics and Applied Mathematics
NUI Galway

7. Power Laws and Scale-Free Graphs

Lecture 18: Preferential Attachment ¶

In the random graph models we have studied so far, a network is generated on a given number of nodes, and edges are randomly added or modified. Here we introduce and study preferential attachment (https://en.wikipedia.org/wiki/Preferential_attachment) models, where a network is grown by adding one node at a time, plus some random edges.
This is a simplified version of a behaviour that can be observed, for example, in citation networks. It turns out that, under suitable circumstances, such a network has a power law degree distribution. A network with a power law degree distribution is called a scale free network (https://en.wikipedia.org/wiki/Scale-free_network).

In [1]:
```
import numpy as np
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
opts = { "with_labels": True, "node_color": 'y' }
```

The Molloy-Reed Criterion

Graphs with a given degree distribution can be analyzed. In particular, a criterion for the existence of a giant component can be formulated in terms of the first and second moments of the degree distribution:

$$\langle k \rangle = \sum_k k p_k, \qquad \langle k^2 \rangle = \sum_k k^2 p_k.$$

Theorem (Molloy-Reed, 1995/1998). Suppose $\underline{p} = (p_0, p_1, \ldots)$ is a degree distribution, i.e., $\sum_k p_k = 1$ and consider the ensemble of all graphs on n nodes with degree distribution $\underline{p}$. Define

$$Q(\underline{p}) := \sum_k k(k-2)p_k = \langle k^2 \rangle - 2\langle k \rangle.$$

Then:

- if $Q(\underline{p}) > 0$ almost every graph in the ensemble contains a **giant component**,
- if $Q(\underline{p}) < 0$ all components are **small**.

- For example, in an ER random graph (models $G(n,m)$ or $G(n,p)$) the moments are related as $\langle k^2 \rangle = \langle k \rangle^2 + \langle k \rangle$, whence $Q(\underline{p}) = \langle k \rangle^2 - \langle k \rangle > 0$ if and only if $\langle k \rangle > 1$, as shown previously.

- In a network with power law degree distribution $p_k \simeq ck^{-\gamma}$ for $2 \leq \gamma \leq 3$, the second moment $\langle k^2 \rangle$ diverges (with $n \to \infty$) and thus $Q(\underline{p}) > 0$: such networks have **giant components**.

- **Characteristic Path Length.** Scale free networks with $2 \leq \gamma \leq 3$ are **ultrasmall**: $L \sim \ln \ln n$ (as opposed to $L \sim \ln n$ in a small world network.)

- **Clustering Coefficient.** The clustering coefficient of a scale-free network depends on a number of parameters ...

Citation Networks.

Citation networks are **directed graphs**. In a citation network, the nodes are **scientific publications**, and the directed arcs represent **citations** between papers (where the citing paper points at the cited paper).

- In addition to the network structure, the nodes in a citation network have a natural order, corresponding to their **time of publication**.
- With respect to that order, citations (almost) always **point backwards** in time.
- As a consequence, a citation network is a DAG (**directed acyclic graph**, contains no loops or directed cycles).

Example. The Book's website has a data set of citations between the 1655 articles that appeared from 1978 to 2004 in Scientometrics (https://en.wikipedia.org/wiki/Scientometrics_(journal)), a journal devoted to the field of bibliometrics and the "Science of Science". To load it into this notebook, read the file as an edge list for a directed graph (`Digraph`), insisting that the nodes are of type `int` . Then copy the edges into an empty directed graph on the nodes $1, 2, \ldots, 1655$, to have the nodes in their natural order.

In [2]:
```
E = nx.read_edgelist("data/scientometrics.net", create_using=nx.DiGraph,
G = nx.DiGraph()
G.add_nodes_from(range(1, max(E.nodes())+1))
G.add_edges_from(E.edges())
```

Let's check the order and the size of the resulting graph.

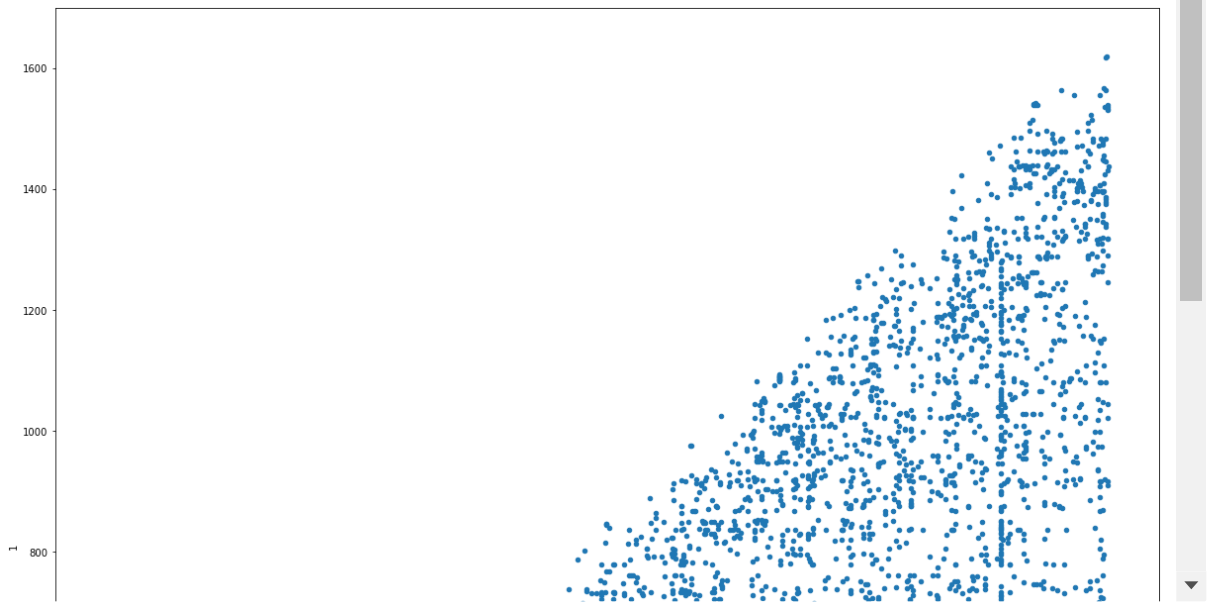
In [3]:
```
n, m = G.order(), G.size()
n, m
```

Out[3]: (1655, 4123)

A scatter plot of the adjacency matrix reveals the DAG nature of the graph: all its nonzero entries lie below the line $i = j$.

In [4]:
```
pd.DataFrame(list(G.edges())).plot.scatter(x = 0, y = 1, figsize = (20,
```

Out[4]: <AxesSubplot:xlabel='0', ylabel='1'>



What are the highest degrees? Inwards and outwards?

In [5]:
```
max_in = max(dict(G.in_degree()).values())
max_in
```

Out[5]: 71

In [6]:
```
max_out = max(dict(G.out_degree()).values())
max_out
```

Out[6]: 156

So there are nodes of exceptionally high degree ...

Next, let's load the time line.

In [7]:
```
years = np.loadtxt("data/scientometrics_paper_year.txt", dtype="int")
years[0]
```

Out[7]: array([1, 1978])

Obviously, this is a list of node/year pairs. Add the year of publication as attribute to each node:

In [8]:
```
for node, year in years:
    G.nodes[node]['year'] = year
```

How many publications per year? (There may be articles without `year` attribute!)

In [9]:
```
from collections import Counter
```

In [10]:
```
counts = Counter(G.nodes[x].get('year') for x in G)
print(counts)
```

```
Counter({None: 284, 1998: 81, 2001: 78, 1996: 77, 1999: 77, 2000: 77,
1992: 74, 2002: 74, 2004: 74, 2003: 73, 1997: 69, 1991: 63, 1994: 59,
1995: 58, 1989: 57, 1993: 52, 1985: 47, 1987: 47, 1990: 45, 1988: 42,
1986: 34, 1982: 23, 1980: 21, 1981: 19, 1983: 19, 1984: 16, 1979: 12,
1978: 3})
```

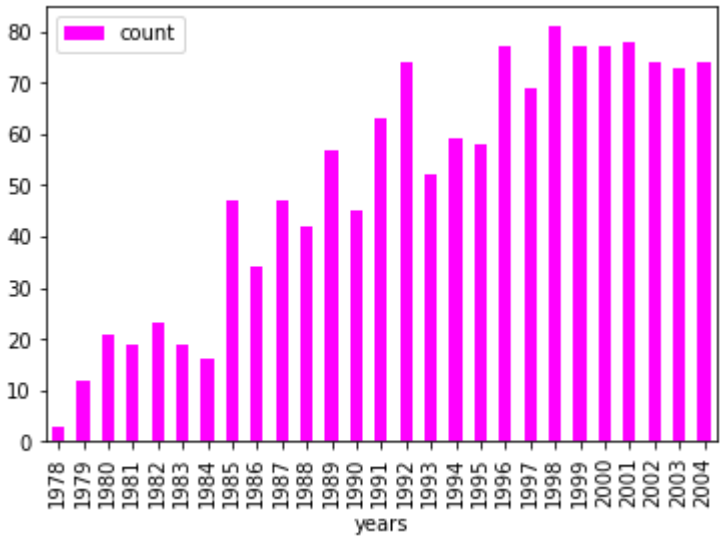
Remove the count of undated articles from the dictionary.

In [11]:
```
counts.pop(None, 0)
```

Out[11]:
```
284
```

In [12]:
```
data = pd.DataFrame(counts.items(), columns=['years', 'count'])
data.plot.bar(x='years', y='count', cmap='spring')
```

Out[12]:
```
<AxesSubplot:xlabel='years'>
```



Identify the node(s) of maximal citation count (in-degree) and show its development over time ...

In [13]:
```
hi = [x for x in G if G.in_degree(x) == max_in]
print(hi)
```

```
[412]
```

In [14]:
```
hi = hi[0]
citing = [x for x in G if hi in G[x]]
print(citing)
```

```
[448, 502, 505, 520, 524, 527, 528, 541, 543, 580, 585, 590, 594, 603,
611, 644, 654, 661, 662, 666, 671, 680, 681, 691, 695, 703, 705, 713,
720, 722, 732, 779, 787, 793, 806, 823, 830, 836, 849, 878, 886, 902,
913, 1003, 1010, 1022, 1023, 1034, 1044, 1127, 1130, 1133, 1149, 1153,
1155, 1181, 1183, 1195, 1200, 1221, 1244, 1266, 1314, 1333, 1396, 146
2, 1480, 1510, 1578, 1582, 1641]
```

In [15]:
```
citing_years = [G.nodes[x].get('year') for x in citing]
print(citing_years)
```

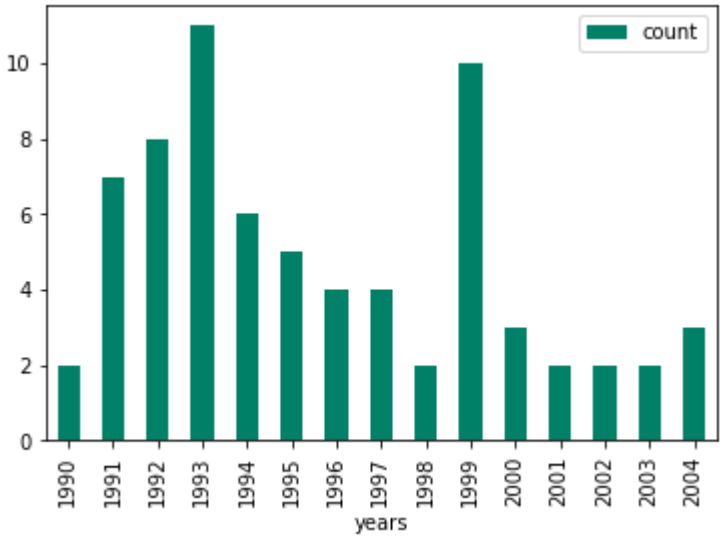
```
[1990, 1990, 1991, 1991, 1991, 1991, 1991, 1991, 1991, 1992, 1992, 199
2, 1992, 1992, 1992, 1992, 1992, 1993, 1993, 1993, 1993, 1993, 1993, 1
993, 1993, 1993, 1993, 1993, 1994, 1994, 1994, 1994, 1994, 1994, 1995,
1995, 1995, 1995, 1995, 1996, 1996, 1996, 1996, 1997, 1997, 1997, 199
7, 1998, 1998, 1999, 1999, 1999, 1999, 1999, 1999, 1999, 1999, 1999, 1
999, 2000, 2000, 2000, 2001, 2001, 2002, 2002, 2003, 2003, 2004, 2004,
2004]
```

In [16]:
```
years = Counter(citing_years)
print(years)
```

```
Counter({1993: 11, 1999: 10, 1992: 8, 1991: 7, 1994: 6, 1995: 5, 1996:
4, 1997: 4, 2000: 3, 2004: 3, 1990: 2, 1998: 2, 2001: 2, 2002: 2, 200
3: 2})
```

In [17]:
```
data = pd.DataFrame(years.items(), columns=['years', 'count'])
data.plot.bar(x='years', y='count', cmap='summer')
```

Out[17]: <AxesSubplot:xlabel='years'>

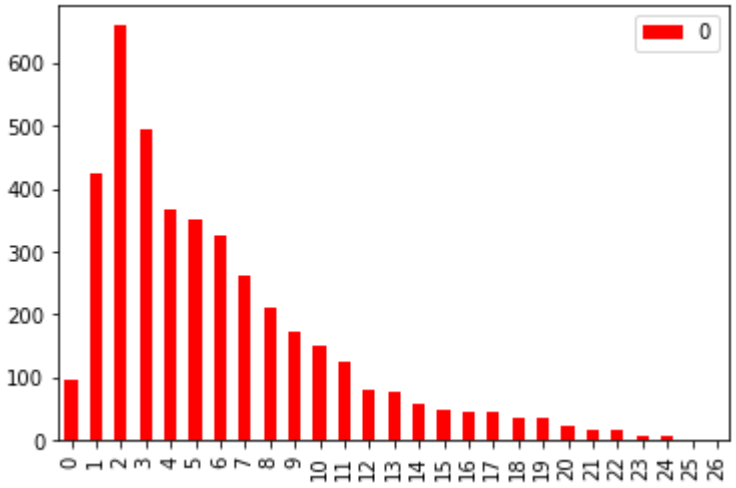


How far back do citations go?

In [18]:
```
span = {}
for e in G.edges():
    years = [G.nodes[n].get('year') for n in e]
    if None not in years:
        s = years[0] - years[1]
        span[s] = span.get(s, 0) + 1
```

In [19]:
```
all = [x[1] for x in sorted(span.items())]
pd.DataFrame(all).plot.bar(colormap='autumn')
```

Out[19]: <AxesSubplot:>



Now let's go back in time one year and recover the state of the citation network at the end of 2003.

In [20]:
```
nodes0 = [x for x in G if G.nodes[x].get('year', 0) < 2004]
G0 = G.subgraph(nodes0)
G0.order(), G0.size()
```

Out[20]:
```
(1581, 3787)
```

Now we can analyse the references in the articles published in 2004 with respect to the in-degree of the cited article in the 2003 network:

- add citation counters as attribute to nodes in `G0`
- then loop over all nodes and determine citations per in_degree ...

In [21]:
```
for x in G0:
    G0.nodes[x]['citations'] = 0

for x in G:
    if G.nodes[x].get('year') == 2004:
        for y in G[x]:
            if y in G0:
                G0.nodes[y]['citations'] += 1
```

Finally, for each in-degree in the 2003 citation graph `G0`, we try and determine how often a node with that many citations in 2003 gets cited on avergage in 2004. For this, we count (in a dict `nr`) the number of nodes of a given degree, and (in a dict `cd`) the total number of citations the nodes of the given degree receive in 2004. Then the average is computed (in a dict `avg`) as the quotient of those two numbers.

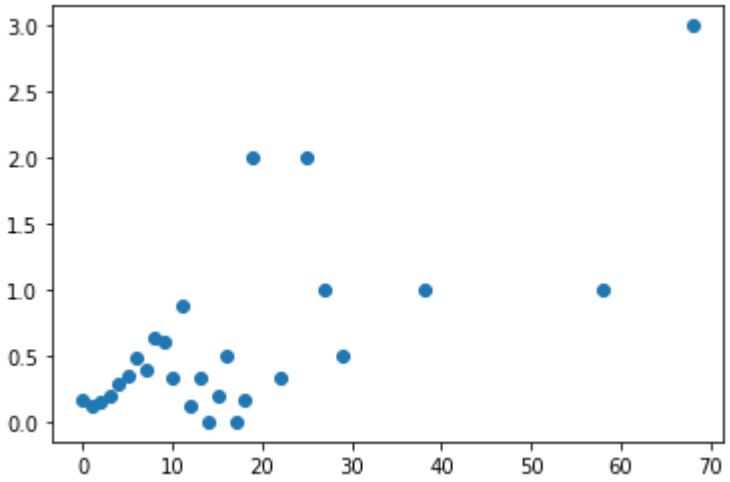
In [22]:
```
nr = {}
cd = {}
for x in G0:
    d = G0.in_degree(x)
    nr[d] = nr.get(d, 0) + 1
    cd[d] = cd.get(d, 0) + G0.nodes[x]['citations']
print(cd)
```

```
{7: 14, 4: 25, 3: 28, 0: 95, 29: 1, 22: 1, 13: 3, 2: 30, 18: 1, 1: 37,
8: 22, 19: 6, 9: 6, 12: 1, 6: 22, 5: 18, 10: 4, 11: 7, 17: 0, 25: 2, 5
8: 1, 14: 0, 15: 1, 16: 1, 68: 3, 38: 1, 27: 1}
```

In [23]:
```
avg = { d: cd[d]/nr[d] for d in cd }
```

In [24]:
```
plt.scatter(avg.keys(), avg.values())
```

Out[24]:
```
<matplotlib.collections.PathCollection at 0x7f9fb195c910>
```



This plot is not a straight line. But still it illustrates the tendency of highly cited nodes to receive even more citations: Roughly, the number of citations that an article receives is proportional to the number of citations it already has. This is a typical rich-get-richer (https://en.wikipedia.org/wiki/The_rich_get_richer_and_the_poor_get_poorer) phenomenon.

Linear Preferential Attachment.

The tendency of new nodes to attach themselves to the most popular nodes, with a probabilty that is proportional to a node's popularity, has many names, and has been discussed (in a biological context) as far back as 1925. These days, the phenomenon is known as **linear preferential attachment**, after Barabási-Albert, who rediscovered it in their analysis of a portion of the WWW in 1999.

Definition (BA-Model). Suppose n, a, b are integers with $0 < b \leq a \ll n$. An (n, a, b)-BA model is a (simple) graph on n nodes, constructed as follows.

1. start with a complete graph on a nodes $\{0, 1, 2, \dots, a-1\}$ (at time $t = 0$)
2. for $t = 1, \dots, n - a - 1$:
    - add new node $x = a - 1 + t$
    - and b links to old nodes i with probability

$$p_{x \to i} = \frac{k_{i,t-1}}{2m_{t-1}},$$

where $k_{i,t}$ is the degree of node i at time t and m_t is the number of edges at time t.

In other words, this is a simple modification of the **random copying** process from Lecture 16, implemented as Python function `powerlaw()`, which we used later to generate a degree sequences with a power law distribution.

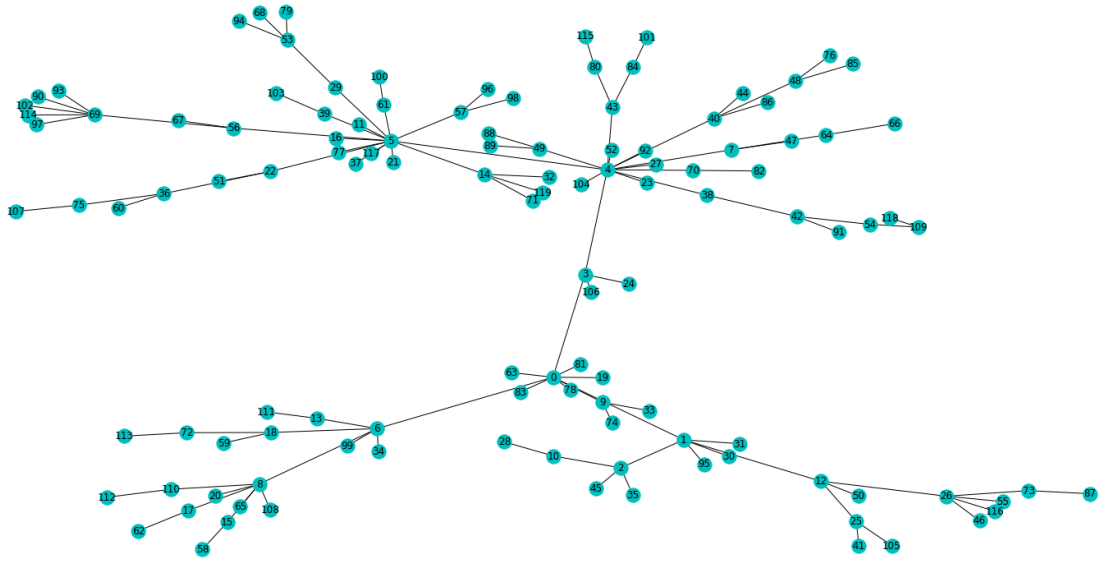
The difference: rather than flipping a coin to decide whether to increase an existing pile or to start a new one, now we do both in one step: start a new node and connect in to an exisiting one, with a probability proportional to its current degree:

In [25]:
```
import random

def preferential(n):
    G = nx.complete_graph(1)
    for i in range(1,n):
        G.add_edge(i, random.choices(*zip(*list(G.degree)))[0])
    return G
```

In [26]:
```
G = preferential(120)
```

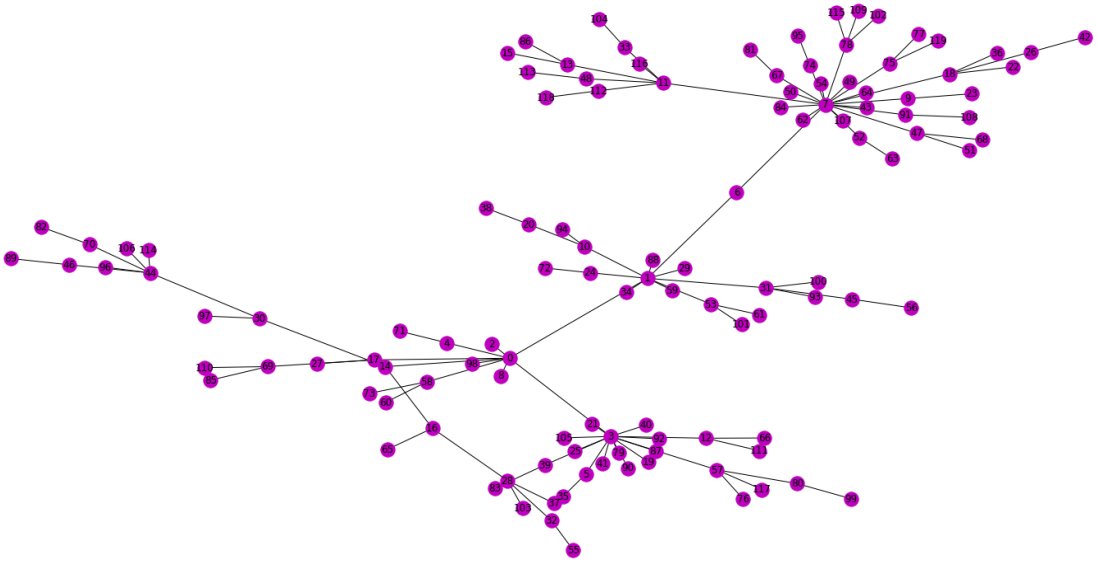
In [27]:
```
opts = { 'with_labels': True, 'node_color': 'c' }
plt.figure(figsize=(20,10))
nx.draw(G, **opts)
```



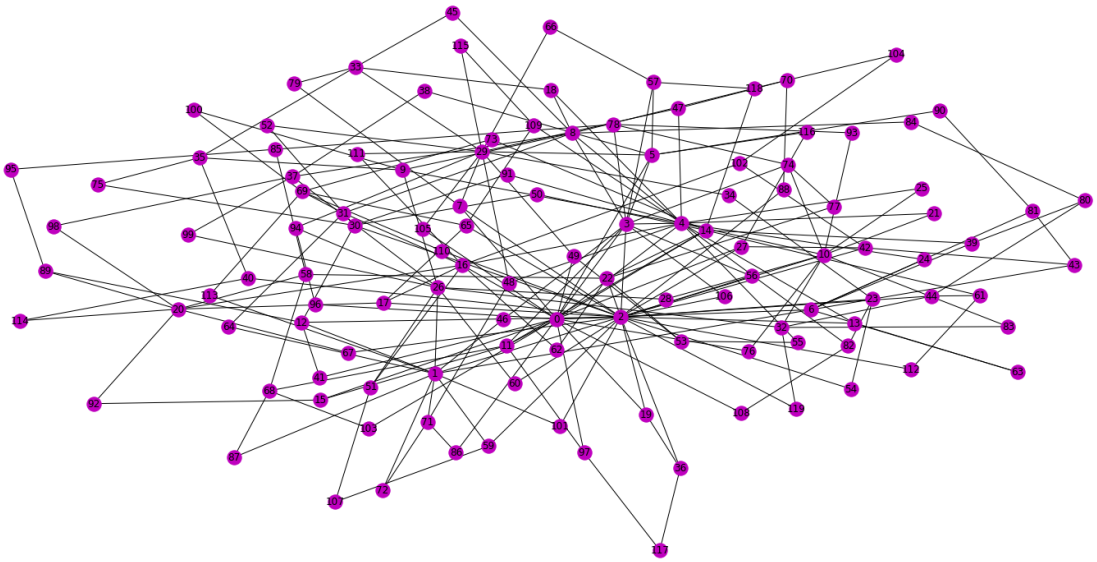
In `networkx`, a BA-model can be generated with the function `nx.barabasi_albert_graph(n, b)` (where $a = b$ in terms of our parameters, and the initial graph is an **empty** graph on the a nodes $\{0, 1, \ldots, a-1\}$).

In [28]:
```
B = nx.barabasi_albert_graph(120, 1)
```

In [29]:
```
opts['node_color'] = 'm'
plt.figure(figsize=(20,10))
nx.draw(B, **opts)
```



In [30]:
```
B = nx.barabasi_albert_graph(120, 2)
plt.figure(figsize=(20,10))
nx.draw(B, **opts)
```

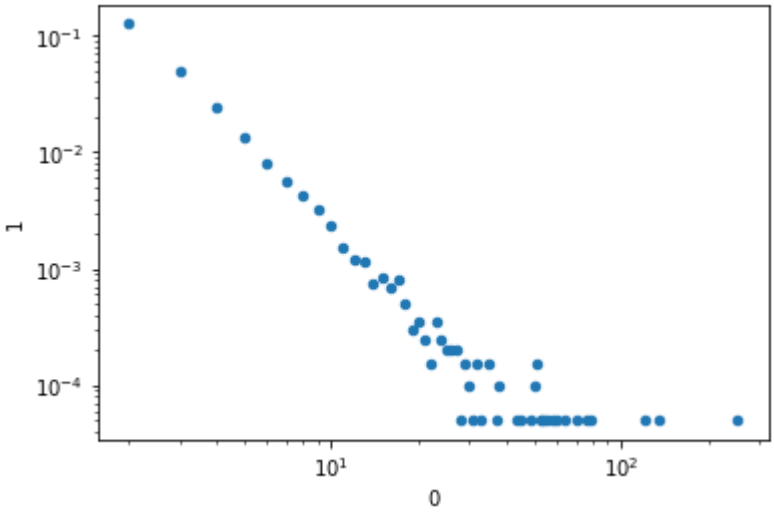


Does such a graph sport a power law degree distribution?

In [31]:
```
B = nx.barabasi_albert_graph(5000, 2)
hist = nx.degree_histogram(B)
m2 = 2*B.size()
data = [(i, v/m2) for i, v in enumerate(hist) if v > 0]
plt.figure(figsize=(20,10))
pd.DataFrame(data).plot.scatter(x = 0, y = 1, loglog=True)
```

Out[31]:
```
<AxesSubplot:xlabel='0', ylabel='1'>

<Figure size 1440x720 with 0 Axes>
```



Fact. An (n, a, b)-BA model (for sufficiently large n) has a power law degree distribution $p_k \simeq ck^{-\gamma}$ with $\gamma = 3$.

More precisely, it can be shown that

$$p_k = \frac{2b(b+1)}{k(k+1)(k+2)} \simeq 2b(b+1)k^{-3}.$$

However, `B` does not have many triangles ...

In [32]:
```
nx.average_clustering(B)
```

Out[32]:
```
0.009408941978781112
```

Code Corner

numpy

- `loadtxt` : [doc] (https://docs.scipy.org/doc/numpy/reference/generated/numpy.loadtxt.html)

`matplotlib`

- `pyplot.scatter` : [doc] (https://matplotlib.org/3.2.1/api/_as_gen/matplotlib.pyplot.scatter.html)

`pandas`

- `DataFrame.plot.bar` : [doc] (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.plot.bar.html)
- `DataFrame.plot.scatter` : [doc] (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.plot.scatter.html)

`networkx`

- `read_edgelist` : [doc] (https://networkx.github.io/documentation/stable/reference/readwrite/generated/networkx.read
- `barabasi_albert_graph` : [doc] (https://networkx.github.io/documentation/stable/reference/generated/networkx.generators.ra

Exercises

1. Modify the function `preferential` above in such a way that it takes an additional parameter `b`, and then starts off with a complete graph on `b` nodes, and for each new node x, adds `b` links from x to old nodes.
2. Produce some evidence, in the form of a suitable scatter plot, that the modified function `preferential` produces a graph with a power law degree distribution.
3. Compute and draw a BA-model graph on `n = 100` nodes with parameter `b = 1`. Compare this graph with the graph `B` above (where `b = 2`).
4. Compute and draw a BA-model graph on `n = 100` nodes with parameter `b = 3`. Compare this graph with the graph `B` above (where `b = 2`).

In [ ]: